Guia de interface — CRUD na tela com fetch

Disciplina: Programação III

Curso: Ensino Médio Integrado — Técnico em Informática

Professor: Elder Francisco Fontana Bernardi

Instituição: IFSul — Campus Passo Fundo

Versão 2 · 27/10/2026

Este guia mostra, num exemplo completo e pronto para rodar, como uma página HTML conversa com uma API Express usando `fetch`: listar, cadastrar, editar e excluir registros sem recarregar a página. No fim, ele indica como aplicar cada padrão às músicas do projeto, sem entregar o código das músicas.

Para que serve

No projeto integrador, "O repertório da turma", os requisitos R2 (cadastrar), R3 (editar), R4 (excluir) e R5 (validar e informar) acontecem **na tela**: a pessoa digita, clica e vê o resultado. A API de músicas já responde; falta a parte da página que chama a API, interpreta a resposta e atualiza o que aparece.

O guia usa um exemplo menor, o **Mural de recados**, em que cada recado tem só um texto. Com um campo só, dá para ver cada padrão sem o ruído das músicas. Depois de ler o mural, vocês aplicam os mesmos padrões ao catálogo, com duas diferenças que a seção "Aplicando às músicas" explica.

O que o guia entrega e o que não entrega:

Entrega	Não entrega
O mural completo, com cada arquivo comentado	O código de cadastro, edição e exclusão de músicas
Os sete padrões que se repetem em qualquer tela com API	A decisão de como organizar o código de vocês
Como conferir cada operação na aba Rede do navegador	A resposta pronta do Percurso para testar e demonstrar
Tabelas e checklists para as músicas	

> **Para quem é cada trilha.** Quem usa a base de retomada (o projeto que acompanha o Roteiro de retomada, disponível no Classroom da disciplina) e quem seguiu o *Guia passo a passo — Karaokê com banco de dados no Supabase* (disponível no Classroom da disciplina) chegam à mesma tela por caminhos diferentes. A seção "Aplicando às músicas" tem uma parte para cada caso.

Rodar o exemplo

O mural está na pasta `mural_recados`, dentro do pacote de materiais de apoio, disponível no Classroom da disciplina. Descompactem o pacote, abram a pasta `mural_recados` no VS Code e, no terminal integrado:

```
npm install
npm start
```

O terminal deve mostrar `Mural no ar: http://localhost:3001`. Abram esse endereço no navegador. A página lista um recado inicial ("Ensaio do karaokê na sexta.") e já aceita cadastrar, editar e excluir.

Os recados ficam num array na memória do servidor. Se vocês pararem o servidor e subirem de novo, a lista volta ao recado inicial. Isso é esperado: o mural ensina a tela, não a persistência. As músicas do projeto, essas sim, ficam no banco.

Árvore de arquivos

```
mural_recados/
├── package.json
├── app.js
└── public/
    ├── index.html
    ├── estilo.css
    └── mural.js
```

Quem usa quem

Arquivo	Usa	Para quê
`public/index.html`	`estilo.css` e `mural.js`	A estrutura da página: formulário, lista, mensagem.
`public/mural.js`	os elementos de `index.html`, pelos IDs; a rota `/api/recados`	Lê o que a pessoa digitou, chama a API e desenha o resultado.
`app.js`	`express`; a pasta `public`	Serve os arquivos da pasta `public` e responde às rotas `/api/recados`.
`package.json`	—	Declara a dependência (`express`) e o comando `npm start`.

O caminho de uma ação, do clique ao resultado: `index.html` mostra o botão, `mural.js` trata o clique e chama `fetch('/api/recados')`, `app.js` recebe a requisição, altera o array e responde, e `mural.js` redesenha a lista com a resposta.

Os cinco arquivos, por inteiro

Os arquivos abaixo são exatamente os do pacote. Leiam cada um uma vez antes de seguir para os padrões; as seções seguintes voltam a trechos específicos.

Arquivo `package.json`

```
{
  "name": "mural-recados",
  "version": "1.0.0",
  "private": true,
  "description": "Exemplo de CRUD na tela com fetch — Programação III, IFSul
Passo Fundo",
  "type": "commonjs",
  "main": "app.js",
  "scripts": { "start": "node app.js" },
  "dependencies": { "express": "5.2.1" },
  "engines": { "node": "⩾22" }
}
```

O `package.json` declara só o `express`, na versão exata, e exige Node 22 ou superior. O comando `npm start` roda `node app.js`.

Arquivo `app.js`

```
// mural_recados/app.js — API de exemplo: recados guardados num array
(memória).
const express = require('express');
const path = require('node:path');

const app = express();
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

let recados = [{ id: 1, texto: 'Ensaio do karaokê na sexta.' }];
let proximoId = 2;

// Lê o texto do corpo; devolve null se faltar ou vier vazio
function lerTexto(body) {
    const texto = typeof body?.texto === 'string' ? body.texto.trim() : '';
    return texto || null;
}

app.get('/api/recados', (req, res) => {
    res.json(recados);
});

app.get('/api/recados/:id', (req, res) => {
    const recado = recados.find((r) => r.id === Number(req.params.id));
    if (!recado) return res.status(404).json({ erro: 'Recado não encontrado' })
;
    res.json(recado);
});

app.post('/api/recados', (req, res) => {
    const texto = lerTexto(req.body);
    if (!texto) return res.status(400).json({ erro: 'Escreva o texto do
recado' });
    const recado = { id: proximoId++, texto };
    recados.push(recado);
    res.status(201).json(recado);
});

app.put('/api/recados/:id', (req, res) => {
    const texto = lerTexto(req.body);
    if (!texto) return res.status(400).json({ erro: 'Escreva o texto do
recado' });
    const recado = recados.find((r) => r.id === Number(req.params.id));
    if (!recado) return res.status(404).json({ erro: 'Recado não encontrado' })
;
    recado.texto = texto;
    res.json(recado);
```

```
});

app.delete('/api/recados/:id', (req, res) => {
    const indice = recados.findIndex((r) => r.id === Number(req.params.id));
    if (indice === -1) return res.status(404).json({ erro: 'Recado não
encontrado' });
    const [removido] = recados.splice(indice, 1);
    res.json({ mensagem: 'Recado removido', recado: removido });
});

const PORT = process.env.PORT || 3001;
app.listen(PORT, (erro) => {
    if (erro) {
        console.error(`Não foi possível usar a porta ${PORT}: ${erro.message}`)
;
        process.exit(1);
    }
    console.log(`Mural no ar: http://localhost:${PORT}`);
});
```

Pontos do `app.js` que importam para a tela:

- `express.json()` converte o corpo JSON que o `fetch` envia em `req.body`. Sem essa linha, `req.body` chega vazio.
- `express.static(...)` serve a pasta `public`. É por isso que `http://localhost:3001` abre o `index.html` sem rota própria.
- Os status seguem o contrato que as músicas também seguem: `200` na leitura, edição e exclusão; `201` no cadastro; `400` quando o texto falta; `404` quando o recado não existe. Todo erro vem como `{ erro: '...' }`. É esse `erro` que a tela mostra.
- `lerTexto` devolve `null` para texto ausente, que não seja `string` ou que fique vazio depois do `trim`. A API valida de novo o que a tela já validou: a tela é uma conveniência, a API é a garantia.
- O fim do arquivo usa `app.listen(PORT, (erro) => {...})`. No Express 5, o segundo argumento recebe o erro de inicialização. O caso mais comum é `EADDRINUSE`: a porta 3001 já está ocupada, em geral por outro `npm start` esquecido num terminal. Em vez de falhar em silêncio, o programa imprime a mensagem e encerra com `process.exit(1)`. Para resolver, fechem o outro processo ou rodem com outra porta (`PORT=3002 npm start` no Linux e no macOS; no PowerShell, `$env:PORT=3002; npm start`).

Arquivo `public/index.html`

```html
<!DOCTYPE html>
<html lang="pt-BR">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mural de recados</title>
    <link rel="stylesheet" href="estilo.css">
</head>
<body>
    <main>
        <h1>Mural de recados</h1>

        <form id="formulario-recado">
            <h2 id="titulo-formulario">Novo recado</h2>
            <label for="texto">Texto do recado</label>
            <input id="texto" name="texto" type="text" maxlength="200" autocomp
lete="off">
            <div class="acoes">
                <button id="salvar" type="submit">Cadastrar</button>
                <button id="cancelar-edicao" type="button" hidden>Cancelar
edição</button>
                <button id="excluir" type="button" disabled>Excluir
selecionado</button>
            </div>
            <p id="mensagem" role="status"></p>
        </form>

        <section>
            <h2>Recados</h2>
            <ul id="lista-recados"></ul>
        </section>
    </main>
    <script src="mural.js"></script>
</body>
</html>
```

Arquivo `public/estilo.css`

```css
/* estilo.css — visual simples do mural; funciona em tela estreita. */
* { box-sizing: border-box; }
body { margin: 0; font-family: 'Segoe UI', Arial, sans-serif; background: #f4f6
f8; color: #1f2933; }
main { max-width: 640px; margin: 0 auto; padding: 16px; }
h1 { color: #1d7a2d; }
form, section { background: #fff; border: 1px solid #d9e2e8; border-radius:
8px; padding: 16px; margin-bottom: 16px; }
label { display: block; font-weight: 600; margin-bottom: 4px; }
input { width: 100%; padding: 10px; font-size: 1rem; border: 1px solid #9aa8b2;
 border-radius: 6px; }
.acoes { display: flex; flex-wrap: wrap; gap: 8px; margin-top: 12px; }
button { padding: 10px 14px; font-size: 1rem; border-radius: 6px; border: 1px
solid #1d7a2d; background: #2f9e41; color: #fff; cursor: pointer; }
button[type="button"] { background: #fff; color: #1d7a2d; }
button:disabled { opacity: 0.5; cursor: not-allowed; }
button:focus-visible, input:focus-visible { outline: 3px solid #1d7a2d; outline
-offset: 2px; }
#mensagem { min-height: 1.4em; margin: 12px 0 0; }
#mensagem.sucesso { color: #1d7a2d; }
#mensagem.erro { color: #b00020; font-weight: 600; }
ul { list-style: none; padding: 0; margin: 0; }
li button { width: 100%; text-align: left; margin-bottom: 6px; background:
#fff; color: #1f2933; border-color: #d9e2e8; }
li.selecionado button { border: 2px solid #1d7a2d; background: #e6f4ed; }
```

O CSS não participa da lógica. Duas linhas importam para os padrões: `#mensagem.sucesso` e `#mensagem.erro`, que pintam a mensagem de verde ou de vermelho conforme a classe que o JavaScript coloca, e `li.selecionado button`, que destaca o recado em edição.

Arquivo `public/mural.js`

```
// public/mural.js — tela do mural: lista, cadastra, edita e exclui recados
pela API.
const formulario = document.getElementById('formulario-recado');
const campoTexto = document.getElementById('texto');
const tituloFormulario = document.getElementById('titulo-formulario');
const botaoSalvar = document.getElementById('salvar');
const botaoCancelar = document.getElementById('cancelar-edicao');
const botaoExcluir = document.getElementById('excluir');
const mensagem = document.getElementById('mensagem');
const lista = document.getElementById('lista-recados');

let idEmEdicao = null;   // null = cadastrando; um número = editando esse
recado

// Mostra uma mensagem; erro = true pinta de vermelho
function mostrarMensagem(texto, erro) {
    mensagem.textContent = texto;
    mensagem.className = erro ? 'erro' : 'sucesso';
}

// Faz a requisição e devolve o JSON. Lança Error em dois casos:
// 1) o servidor respondeu com erro (400, 404, 500...)
// 2) não houve resposta (servidor parado, sem rede)
async function chamarApi(caminho, opcoes) {
    let resposta;
    try {
        resposta = await fetch(caminho, opcoes);
    } catch {
        throw new Error('Sem conexão com o servidor. Confiram se ele está
rodando.');
    }
    const dados = await resposta.json().catch(() => null);
    if (!resposta.ok) {
        throw new Error(dados?.erro || 'O servidor respondeu com erro ' +
resposta.status);
    }
    return dados;
}

// Desabilita os botões enquanto uma requisição está em andamento
function definirOcupado(ocupado) {
    botaoSalvar.disabled = ocupado;
    botaoCancelar.disabled = ocupado;
    botaoExcluir.disabled = ocupado || idEmEdicao === null;
}

// Busca os recados e redesenha a lista
```

```
async function carregarRecados() {
    const recados = await chamarApi('/api/recados');
    lista.innerHTML = '';
    for (const recado of recados) {
        const item = document.createElement('li');
        const botao = document.createElement('button');
        botao.type = 'button';
        botao.textContent = recado.texto;   // textContent: o texto aparece
como texto, nunca como HTML
        botao.addEventListener('click', () => selecionarRecado(recado));
        if (recado.id === idEmEdicao) item.className = 'selecionado';
        item.appendChild(botao);
        lista.appendChild(item);
    }
}

// Clicar num recado: o formulário passa para o modo edição
function selecionarRecado(recado) {
    idEmEdicao = recado.id;
    campoTexto.value = recado.texto;
    tituloFormulario.textContent = 'Editando recado ' + recado.id;
    botaoSalvar.textContent = 'Salvar alteração';
    botaoCancelar.hidden = false;
    botaoExcluir.disabled = false;
    mostrarMensagem('', false);
    carregarRecados().catch((erro) => mostrarMensagem(erro.message, true));   /
/ redesenha o destaque
}

// Volta o formulário para o modo cadastro
function voltarParaCadastro() {
    idEmEdicao = null;
    formulario.reset();
    tituloFormulario.textContent = 'Novo recado';
    botaoSalvar.textContent = 'Cadastrar';
    botaoCancelar.hidden = true;
    botaoExcluir.disabled = true;
}

// Submit do formulário: POST (cadastro) ou PUT (edição)
async function salvarRecado(evento) {
    evento.preventDefault();   // impede o navegador de recarregar a página
    const texto = campoTexto.value.trim();
    if (!texto) {
        mostrarMensagem('Escreva o texto do recado.', true);
        campoTexto.focus();
        return;
```

```
    }

    const editando = idEmEdicao !== null;
    const caminho = editando ? '/api/recados/' + idEmEdicao : '/api/recados';
    definirOcupado(true);
    try {
        const recado = await chamarApi(caminho, {
            method: editando ? 'PUT' : 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({ texto })
        });
        voltarParaCadastro();
        await carregarRecados();
        mostrarMensagem((editando ? 'Recado alterado' : 'Recado cadastrado') +
' (id ' + recado.id + ').', false);
    } catch (erro) {
        mostrarMensagem(erro.message, true);   // o texto digitado continua no
campo
    } finally {
        definirOcupado(false);
    }
}

// Botão Excluir: confirma antes de enviar o DELETE
async function excluirRecado() {
    if (idEmEdicao === null) return;
    const confirmou = window.confirm('Excluir o recado ' + idEmEdicao + '?');
    if (!confirmou) {
        mostrarMensagem('Exclusão cancelada.', false);
        return;
    }
    definirOcupado(true);
    try {
        const resultado = await chamarApi('/api/recados/' + idEmEdicao, { metho
d: 'DELETE' });
        voltarParaCadastro();
        await carregarRecados();
        mostrarMensagem('Recado ' + resultado.recado.id + ' excluído.', false);
    } catch (erro) {
        mostrarMensagem(erro.message, true);
    } finally {
        definirOcupado(false);
    }
}

formulario.addEventListener('submit', salvarRecado);
botaoCancelar.addEventListener('click', () => {
```

```
    voltarParaCadastro();
    mostrarMensagem('Edição cancelada.', false);
});
botaoExcluir.addEventListener('click', excluirRecado);

carregarRecados().catch((erro) => mostrarMensagem(erro.message, true));
```

Padrão 1 — HTML e JavaScript se encontram pelos IDs

O JavaScript não "enxerga" o HTML por posição nem por aparência. Ele procura cada elemento por um **ID**, com `document.getElementById`, e guarda o resultado numa constante. Se o ID do HTML e o texto procurado no JavaScript não forem idênticos, a busca devolve `null` e a primeira linha que usar a constante quebra com `Cannot read properties of null`.

As primeiras linhas de `mural.js` fazem esse encontro:

ID no `index.html`	Constante em `mural.js`	Papel
`formulario-recado`	`formulario`	O `<form>`; recebe o evento `submit` e o `reset()`.
`texto`	`campoTexto`	O campo de texto; `value` lê e escreve o que está digitado.
`titulo-formulario`	`tituloFormulario`	Muda entre "Novo recado" e "Editando recado N".
`salvar`	`botaoSalvar`	Botão `submit`; o texto alterna entre "Cadastrar" e "Salvar alteração".
`cancelar-edicao`	`botaoCancelar`	Aparece só no modo edição (atributo `hidden`).
`excluir`	`botaoExcluir`	Começa desabilitado; só vale com um recado selecionado.
`mensagem`	`mensagem`	Onde sucesso e erro aparecem; tem `role="status"` para leitores de tela.
`lista-recados`	`lista`	O `<ul>` que a função de listar preenche.

A ordem do `<script>` importa. No `index.html`, a tag `<script src="mural.js">` fica no **fim do `body`**, depois de todos os elementos. O navegador lê o HTML de cima para baixo: se o script viesse no `head`, ele rodaria antes de o formulário existir, e todos os `getElementById` devolveriam `null`.

Como conferir. Abram o console do navegador (F12, aba Console) e digitem `document.getElementById('texto')`. Se aparecer o elemento, o ID existe; se aparecer `null`, há diferença entre HTML e JavaScript (maiúscula, hífen, letra trocada).

> **Pergunte ao agente.** *"Explique o que acontece se eu esquecer o evento.preventDefault() no submit. Antes de responder, peça que eu preveja."*

Padrão 2 — Listar e desenhar

A função `carregarRecados` faz três coisas, sempre nesta ordem: **buscar** os dados na API, **limpar** a lista antiga e **desenhar** um item para cada registro.

```
buscar:   const recados = await chamarApi('/api/recados');
limpar:   lista.innerHTML = '';
desenhar: para cada recado, criar <li> + <button>, preencher, anexar à lista
```

Pontos que valem atenção:

- A função é `async` porque `fetch` é assíncrono. O `await` faz a função esperar a resposta antes de seguir.
- `lista.innerHTML = ''` apaga os itens antigos. Sem isso, cada chamada acrescentaria a lista inteira de novo ao final da anterior.
- Cada item é criado com `document.createElement` e preenchido com **`textContent`**, não com `innerHTML`.

Por que `textContent` e não `innerHTML` para dados. O `innerHTML` interpreta o texto como HTML. O `textContent` mostra o texto como texto. Para dados que vêm de uma pessoa (o recado, o nome de uma música), só o `textContent` é seguro.

Teste do negrito. Cadastrem o recado `<b>negrito</b>`. Com `textContent`, a lista mostra exatamente `<b>negrito</b>`, com os sinais de menor e maior. Se vocês trocassem a linha por `botao.innerHTML = recado.texto`, a lista mostraria a palavra "negrito" em negrito, porque o navegador trataria o texto como marcação. Com um texto como `<img src=x onerror=...>`, a mesma troca executaria código de quem digitou. Isso se chama injeção de HTML (XSS), e o `textContent` evita o problema sem esforço.

Observação sobre `selecionarRecado`. Cada clique num recado chama `carregarRecados()` de novo, só para redesenhar qual item está destacado. Funciona, mas gasta uma requisição por clique. Uma alternativa mais simples seria trocar a classe `selecionado` direto nos elementos da lista, sem ir à API. O mural mantém o recarregamento porque ele mostra, num único lugar, como a lista é sempre desenhada a partir do estado (`idEmEdicao`). Vocês podem escolher outro caminho nas músicas, desde que a lista e o destaque não fiquem fora de sincronia.

Padrão 3 — Cadastrar

O cadastro é o caso base: `salvarRecado`, quando `idEmEdicao` é `null`, envia um `POST`. O caminho, linha a linha:

1. **`evento.preventDefault()`** é a primeira linha. O `submit` de um formulário recarrega a página por padrão. Sem esta linha, a página recarrega antes de a resposta da API chegar, a mensagem some e a lista é redesenhada do zero.
2. **`campoTexto.value.trim()`** lê o texto e tira os espaços das pontas. Um texto só com espaços vira vazio.
3. **Validação antes de enviar.** Se o texto estiver vazio, a função mostra a mensagem de erro, devolve o foco ao campo com `focus()` e termina com `return`. Nenhuma requisição é feita. Isso atende ao R5 no lado da tela.
4. **O `fetch` do POST** leva três opções:

Opção	Valor	Por quê
`method`	`'POST'`	Cadastrar.

Opção	Valor	Por quê
`headers`	`{ 'Content-Type': 'application/json' }`	Avisa a API de que o corpo é JSON. Sem este cabeçalho, o `express.json()` ignora o corpo e a API responde `400`.
`body`	`JSON.stringify({ texto })`	O `fetch` só envia texto. `JSON.stringify` transforma o objeto em texto JSON.

5. **Status `201`.** A API responde `201 Created` com o recado criado. Quem decide se deu certo é `response.ok` (veja o Padrão 4), que vale para qualquer status de `200` a `299`.
6. **Ler o `id` da resposta.** O recado devolvido traz o `id` que o servidor atribuiu. O mural usa esse `id` na mensagem: "Recado cadastrado (id 2)." Esse número é a prova de que o registro foi criado pela API, não só desenhado na tela. Nas músicas, é o `id` que o R2 pede que vocês anotem.
7. **Voltar ao cadastro e recarregar.** `voltarParaCadastro()` limpa o formulário e `await carregarRecados()` redesenha a lista com o novo registro.

Dois detalhes de ordem. A mensagem de sucesso só é escrita **depois** de `chamarApi` retornar sem lançar erro. E a mensagem vem **depois** do recarregamento da lista. Por isso, se a gravação funcionou mas o recarregamento falhou (queda de conexão entre as duas chamadas), a tela mostra a mensagem de falha de conexão, embora o recado tenha sido gravado. A mensagem não está errada sobre o que falhou (a atualização da lista), mas pode confundir. Quando uma mensagem de erro aparece logo depois de um salvamento, abram a aba Rede e leiam a ordem das requisições: se o `POST` aparece com `201`, o registro existe e só a lista não atualizou.

Padrão 4 — Erro HTTP × falha de conexão

O `fetch` tem uma regra que surpreende quem vem de outras linguagens: **ele só lança exceção quando não consegue falar com o servidor.** Se o servidor respondeu, mesmo com `400`, `404` ou `500`, o `fetch` termina normalmente e entrega uma resposta cujo `response.ok` é `false`. Quem confere o status é o seu código.

A função `chamarApi` junta os dois casos num só: lança um `Error` com uma mensagem que a tela pode mostrar. Linha a linha:

```
async function chamarApi(caminho, opcoes) {
    let resposta;                                  // declarada fora do try para
valer depois dele
    try {
        resposta = await fetch(caminho, opcoes);   // falha aqui = sem resposta
nenhuma
    } catch {
        throw new Error('Sem conexão com o servidor. Confiram se ele está
rodando.');
    }
    const dados = await resposta.json().catch(() ⇒ null);   // corpo não é
JSON? vira null
    if (!resposta.ok) {                            // o servidor respondeu, mas
com erro
        throw new Error(dados?.erro || 'O servidor respondeu com erro ' +
resposta.status);
    }
    return dados;                                  // só chega aqui se deu certo
}
```

- O `try/catch` em volta do `fetch` captura apenas a **falha de conexão**.
- `resposta.json().catch(() ⇒ null)` evita um segundo erro quando o corpo da resposta não é JSON (por exemplo, uma página de erro em HTML). Nesse caso, `dados` fica `null`.
- `dados?.erro` usa o campo `erro` que a API manda. Se ele não existir, a mensagem cai no texto genérico com o status.
- Quem chama `chamarApi` usa um `try/catch` só, e trata os dois casos do mesmo jeito: `mostrarMensagem(erro.message, true)`.

Situação, `fetch` e tela

Situação	O que o `fetch` faz	O que a tela mostra
`400` — texto vazio que passou pela tela	Termina normal; `response.ok` é `false`	A mensagem da API: "Escreva o texto do recado", em vermelho
`404` — recado já excluído em outra aba	Termina normal; `response.ok` é `false`	"Recado não encontrado", em vermelho
`500` — erro interno no servidor	Termina normal; `response.ok` é `false`	A mensagem de `erro`, se o servidor mandar; senão "O servidor respondeu com erro 500"
Servidor parado ou sem rede	**Lança exceção** (o `catch` do `try` interno)	"Sem conexão com o servidor. Confiram se ele está rodando.", em vermelho
`200` ou `201`	Termina normal; `response.ok` é `true`	Mensagem de sucesso, em verde

A regra: a tela só anuncia sucesso depois da resposta `ok`. Na prática, a linha `mostrarMensagem('Recado cadastrado...', false)` fica **depois** do `await chamarApi(...)`, nunca antes. Se ela viesse antes, uma queda de conexão deixaria a tela dizendo "cadastrado" para um registro que não existe. É exatamente o erro que o R5 quer evitar.

> **Pergunte ao agente.** *"Qual a diferença entre a resposta ter response.ok falso e o fetch lançar uma exceção? Dê um exemplo de cada situação no meu projeto."*

Padrão 5 — Selecionar e editar

Editar não é uma tela nova. É o **mesmo formulário** em outro modo. Uma única variável de estado, `idEmEdicao`, diz em qual modo ele está:

idEmEdicao	Modo	Título	Botão	Cancelar	Excluir
`null`	Cadastro	Novo recado	Cadastrar	escondido	desabilitado
um número	Edição	Editando recado N	Salvar alteração	visível	habilitado

As três funções que mexem nesse estado:

- **`selecionarRecado(recado)`** entra no modo edição. Guarda o `id`, copia o texto para o campo, troca título e botão, mostra "Cancelar edição" e habilita "Excluir selecionado". Ao final, recarrega a lista para redesenhar o destaque (a observação do Padrão 2).
- **`voltarParaCadastro()`** faz o caminho inverso: `idEmEdicao = null`, `formulario.reset()` e os textos originais. É usada depois de salvar, depois de excluir e ao cancelar.
- **`salvarRecado`** decide sozinha entre criar e alterar:

  ```
  const editando = idEmEdicao !== null;
  const caminho  = editando ? '/api/recados/' + idEmEdicao : '/api/recados';
  method: editando ? 'PUT' : 'POST'
  ```

  O mesmo `submit` serve aos dois modos. O que muda é o método (`PUT` ou `POST`) e o endereço (com ou sem `/id`). É por isso que a variável `idEmEdicao` precisa estar certa na hora de salvar: se ela ficar `null` no modo edição, o `submit` cria um registro novo em vez de alterar o existente.

O botão **Cancelar edição** chama `voltarParaCadastro()` e informa "Edição cancelada." Nada é enviado à API.

Com a API fora do ar, o texto não se perde. No `catch` de `salvarRecado`, a função não chama `voltarParaCadastro()`. O texto digitado continua no campo e o modo de edição continua ativo, para a pessoa tentar de novo.

> **Pergunte ao agente.** *"Estou no modo edição e, ao salvar, é criada uma música nova em vez de alterar a existente. Que perguntas devo me fazer para achar o erro?"*

Padrão 6 — Excluir com confirmação

`excluirRecado` segue uma ordem fixa:

1. **Exigir seleção.** Sem `idEmEdicao`, a função termina na primeira linha. O botão já fica desabilitado, e esta verificação é a segunda barreira.

2. **Confirmar.** `window.confirm('Excluir o recado N?')` abre uma caixa com OK e Cancelar e devolve `true` ou `false`. Se for `false`, a função mostra "Exclusão cancelada." e termina **sem enviar nada**. Esse é o comportamento que o R4 pede: cancelar mantém o registro.
3. **Enviar o `DELETE`** para `/api/recados/` + o `id`. Não há corpo.
4. **Atualizar.** Depois de uma resposta `ok`, `voltarParaCadastro()` zera a seleção, `await carregarRecados()` redesenha a lista sem o registro e a mensagem usa o que a API devolveu: `resultado.recado.id`. A API responde `{ mensagem, recado }`; para as músicas, o contrato é `{ mensagem, musica }`.

Observação sobre o foco. Durante a exclusão, `definirOcupado(true)` desabilita o botão Excluir, e no `finally` ele continua desabilitado (porque `voltarParaCadastro()` já zerou a seleção). O botão que tinha o foco deixou de aceitar foco, e quem navega pelo teclado perde a posição na página. O mural não corrige isso. Se vocês quiserem tratar, o caminho é mandar o foco para o campo de texto depois de excluir. Vale a pena fazer nas músicas se o tempo permitir; não é exigido pelos requisitos.

> **Pergunte ao agente.** *"Revise a minha função de exclusão (colada abaixo). Aponte a linha do problema e explique, sem reescrever a função."*

Padrão 7 — Evitar envio duplicado

Se a pessoa clicar duas vezes em "Cadastrar" antes de a resposta chegar, duas requisições partem e dois registros são criados. A defesa é desabilitar os botões enquanto a requisição está em andamento.

`definirOcupado(ocupado)` faz isso em um lugar só:

```
botaoSalvar.disabled   = ocupado;
botaoCancelar.disabled = ocupado;
botaoExcluir.disabled  = ocupado || idEmEdicao === null;
```

A última linha tem uma sutileza: ao terminar (`ocupado` igual a `false`), o botão Excluir só volta a ficar habilitado se ainda houver um recado selecionado.

O padrão fica completo com o `finally`:

```
definirOcupado(true);
try {
    ...await chamarApi(...)...
} catch (erro) {
    mostrarMensagem(erro.message, true);
} finally {
    definirOcupado(false);     // roda com sucesso OU com erro
}
```

O `finally` roda sempre, inclusive quando o `catch` é executado. Se a liberação estivesse só no fim do `try`, um erro de rede deixaria os botões desabilitados para sempre e a pessoa teria de recarregar a página. É esse o sintoma "o botão Excluir continua desabilitado" da seção "Se deu errado".

Conferir na aba Rede (DevTools)

A tela mostra o que o JavaScript decidiu mostrar. A aba **Rede** (em inglês, *Network*) mostra o que realmente aconteceu entre o navegador e a API. É a fonte da evidência para o Percurso para testar e demonstrar.

Abrir. Com a página do mural aberta, apertem F12 e escolham a aba Rede. Marquem o filtro **Fetch/XHR** para ver só as chamadas à API. Recarreguem a página (F5) para a lista de requisições começar do zero.

O que ler em cada requisição. Cliquem numa linha da lista:

Onde olhar	O que mostra
Coluna **Método** (ou aba Cabeçalhos)	`GET`, `POST`, `PUT` ou `DELETE`
Coluna **Nome** / URL	O endereço chamado, como `/api/recados/2`
Coluna **Status**	`200`, `201`, `400`, `404`...
Aba **Carga** (*Payload*)	O que a página enviou no corpo (o JSON de `JSON.stringify`)
Aba **Resposta** (*Response*)	O que a API devolveu, como texto JSON

Onde ver o `id` criado (evidência do R2). Cadastrem um recado e procurem a requisição `POST` para `/api/recados`. Na aba Resposta, o JSON traz `id` e `texto`. Esse `id`, junto com o status `201`, mostra que a API criou o registro. Anotem o `id`: o Percurso pede que vocês consultem o registro de novo depois.

Falha controlada (R5).

1. Com a página aberta e funcionando, voltem ao terminal onde o `npm start` roda e apertem `Ctrl+C`. O servidor para; a página continua aberta.
2. Na página, digitem um recado e cliquem em "Cadastrar".
3. Leiam a mensagem. Ela deve ser a de falha de conexão ("Sem conexão com o servidor...") e **não** pode dizer que o recado foi cadastrado. Na aba Rede, a requisição aparece em vermelho, sem status.
4. Reiniciem com `npm start`, recarreguem a página e confiram a lista. Como os recados vivem na memória, só o recado inicial volta.

Esse é o mesmo teste da "Falha controlada local" do enunciado do projeto. Nas músicas, a diferença é que a lista volta com os dados do banco.

Aplicando às músicas (sem código)

Esta seção diz **o que muda** do mural para o catálogo e **o que conferir** em cada requisito. Ela não traz o código das músicas: a escrita é de vocês, com os padrões acima.

O que muda do mural para as músicas

Ponto	Mural de recados	Músicas
Campos do formulário	Um (`texto`)	Dois (`nome` e `artista`), ambos obrigatórios

Ponto	Mural de recados	Músicas
Corpo enviado no POST e no PUT	`{ texto }`	`{ nome, artista }`, os dois como **texto**
O que a API devolve para o artista	—	Um **objeto** `{ id, nome }`. Para preencher o campo no modo edição, usem `artista.nome`. Usar `artista` direto escreve `[object Object]` no campo
Editar o artista	—	Muda o **vínculo desta música**: a música passa a apontar para outro artista (que a API encontra ou cria pelo nome). Os outros registros do artista antigo não são renomeados
Lista	Só o texto	Cada item traz também `totalPartes`
Música sem partes	—	É válida. Ao selecioná-la, a tela informa que não há partes e mantém **Tocar** indisponível
Resposta do DELETE	`{ mensagem, recado }`	`{ mensagem, musica }`
Validação na tela	Um campo vazio	Nome vazio **ou** artista vazio: cada um deve ser recusado, e a mensagem precisa dizer qual

Checklist por requisito

Os testes abaixo vêm do "Percurso para testar e demonstrar" do enunciado de "O repertório da turma". Em todos, usem uma música de teste sem partes e preservem a música que o player usa na demonstração.

Requisito	O que a tela precisa fazer	Como conferir
R2 — Cadastrar	Enviar nome e artista (`POST /api/musicas`); mostrar o resultado; recarregar a lista	Cadastrar uma música de teste. Na aba Rede, ver `201` e anotar o `id` da resposta. Consultar a música de novo. Ao selecioná-la, a tela informa que não há partes e Tocar fica indisponível
R3 — Editar	Preencher o formulário ao selecionar; enviar `PUT /api/musicas/:id` com o `id` em edição; manter o `id`	Mudar nome ou artista da música de teste. A lista e a resposta da API mostram a mudança; o `id` é o mesmo. Reiniciar o servidor e consultar de novo: o mesmo `id` com os dados novos
R4 — Excluir	Exigir seleção; pedir confirmação; só então enviar `DELETE /api/musicas/:id`	Iniciar a exclusão e **cancelar**: a consulta seguinte mostra a música ainda cadastrada. Iniciar de novo e **confirmar**: a consulta seguinte não a mostra
R5 — Validar e informar	Recusar campos vazios antes de enviar; mostrar erro em vermelho e sucesso em verde; distinguir erro da API de falha de conexão	Tentar cadastrar com nome vazio e com artista vazio: a tela informa o problema e não anuncia cadastro. Parar o servidor local, tentar atualizar a lista ou cadastrar: a mensagem de erro aparece, nunca a de sucesso. Reiniciar e consultar a lista

Quem usa a base de retomada

Quem usa a base de retomada (disponível no Classroom da disciplina) não escreve a página inteira. Todo o trabalho acontece em **`public/admin.js`**, que já está ligado ao HTML e traz comentários `TODO` para cadastro, edição e exclusão. O `catalogo.js` já oferece:

Item pronto	O que faz
`consultarApi(caminho, opcoes)`	Faz o `fetch`, devolve o JSON em caso de sucesso e **lança** `Error` tanto para erro HTTP quanto para falha de rede. É o equivalente do `chamarApi` do mural
`carregarMusicas()`	Busca as músicas e redesenha a lista (o equivalente de `carregarRecados`)
`limparSelecao()`	Desfaz a seleção atual
`mostrarMensagem(id, texto, erro)`	Escreve uma mensagem no elemento de ID `id`; `erro` pinta como erro. Atenção: recebe o **ID do elemento** como primeiro argumento, ao contrário do mural
`definirCatalogoOcupado(bool)`	Trava e destrava a lista enquanto uma operação está em andamento
`estadoCatalogo.musicaSelecionada`	O detalhe da música selecionada, ou `null`
evento `musica-selecionada`	Disparado ao selecionar ou limpar; o detalhe vem em `evento.detail`, ou `null`

Os IDs do formulário da base: `formulario-musica`, `nome`, `artista`, `salvar-musica`, `excluir-musica`, `nova-musica`, `titulo-formulario` e `estado-formulario`.

Na correspondência com os padrões: o `idEmEdicao` do mural é o ID da música selecionada; o `voltarParaCadastro` tem o papel de preparar um novo cadastro (limpar seleção e campos); o `definirOcupado` combina a trava dos botões do formulário com `definirCatalogoOcupado`. Os controles de escrita vêm desabilitados no HTML: habilitar o botão não basta, é preciso integrar a operação e tratar a resposta.

Quem seguiu o guia do Supabase

Quem seguiu o *Guia passo a passo — Karaokê com banco de dados no Supabase* tem a página em `public/index.html` e o código em `public/player.js` (IDs `listaMusicas`, `nomeMusica`, `artista`, `tag`, `letra`, `contador`, `btnTocar` e `mensagem`). O caminho para as operações de escrita:

1. Acrescentar ao `public/index.html` um `<form>` com os campos `nome` e `artista`, um elemento de mensagem e os botões (cadastrar, cancelar edição e excluir). Cuidado com os IDs: `artista` e `mensagem` **já existem** no player, e dois elementos com o mesmo ID quebram o `getElementById`. Escolham nomes novos para o formulário.
2. Criar o arquivo `public/admin.js` e incluí-lo no `index.html` **depois** do `player.js`, ambos no fim do `body`.
3. Para recarregar a lista depois de cada operação, chamar `carregarPlaylist()`, a função do `player.js` que desenha a lista.
4. Aplicar os Padrões 3 a 7 com o mesmo formato de chamada do mural, trocando `/api/recados` por `/api/musicas`.

Se deu errado

Sintoma	Causa provável	O que fazer
A página recarrega ao salvar e a mensagem some	Faltou `evento.preventDefault()` no `submit`, ou o listener foi ligado ao botão em vez de ao formulário	Conferir a primeira linha da função do `submit` e `formulario.addEventListener('submit', ...)` (Padrão 3)

Sintoma	Causa provável	O que fazer
A tela diz "cadastrado" com o servidor parado	A mensagem de sucesso é escrita antes da resposta, ou o `fetch` é chamado sem tratar a exceção	Conferir que o sucesso só é escrito depois do `await` que retornou sem lançar erro (Padrão 4)
O campo artista mostra `[object Object]`	O campo recebeu o objeto `artista` em vez de `artista.nome`	Preencher com `musica.artista.nome` (a tabela "O que muda")
A lista não atualiza depois de salvar	Faltou chamar (ou esperar) o recarregamento da lista depois do salvamento	Chamar a função de listar com `await` depois da operação. Conferir na aba Rede se o `GET` saiu depois do `POST`
O botão Excluir continua desabilitado	O `definirOcupado(false)` não roda em caso de erro, ou a seleção foi zerada	Colocar a liberação dentro do `finally` (Padrão 7) e conferir o estado da seleção
`Cannot read properties of null` na primeira linha	ID do JavaScript diferente do ID do HTML, ou `<script>` antes dos elementos	Conferir o ID no console com `getElementById` e a posição do `<script>` (Padrão 1)
A API responde `400` mesmo com o campo preenchido	Faltou o cabeçalho `Content-Type: application/json` ou o `JSON.stringify`	Conferir a aba Rede, aba Carga: o corpo deve ser um JSON com os campos
`EADDRINUSE` ao subir o servidor	A porta já está ocupada por outro `npm start`	Fechar o outro processo ou usar outra porta (seção "Os cinco arquivos")